Encoder–decoder

Đọc một nguồn, viết một chuỗi đích

Đầu ra không chỉ một nhãn

Dịch một câu, rút gọn bản tin, mô tả một ảnh.

Nguồn và đích có thể khác độ dài, từ vựng, loại dữ liệu.

Nguồn đã có.

Đích đúng chưa có khi inference.

Model phải viết từng bước có điều kiện.

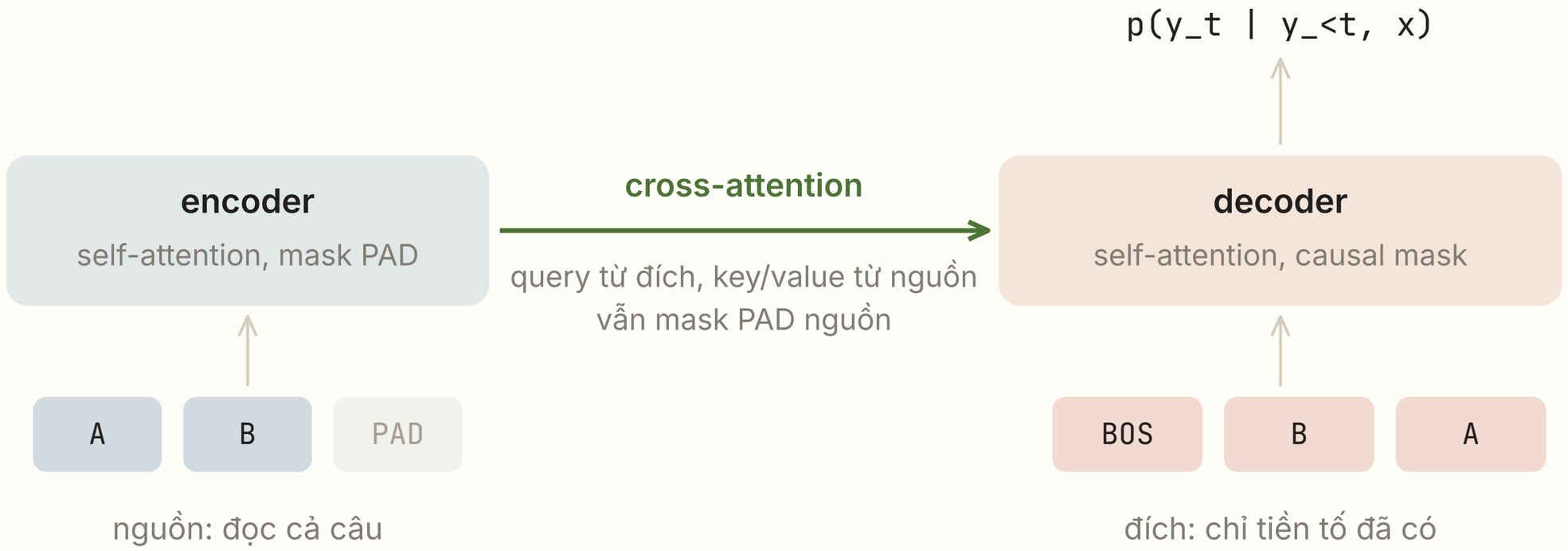
Mục tiêu buổi này

- Tách encoder, decoder, cross-attention và mask từng khối.
- Căn input và label cho teacher forcing; thư viện tự dịch label.
- Đọc loss từng vị trí, so với free-running.
- Giải mã bằng beam; đọc BLEU, ROUGE đúng giới hạn.

01

Hai luồng thông tin

Encoder đọc nguồn, decoder viết đích



Ba chỗ dùng attention trong transformer encoder–decoder: self-attention của encoder mask PAD nguồn, self-attention của decoder có causal mask, cross-attention lấy query từ đích và key, value từ nguồn, vẫn mask PAD nguồn.

Sinh có điều kiện

$$p(y \mid x) = \prod_{t} p(y_t \mid y_{<t}, x)$$

x: nguồn đã biết; y: chuỗi đích cần sinh.

Encoder: hai chiều trên nguồn đã có.

Decoder: causal self-attention trên đích.

Ba chỗ attention, ba mask

Encoder self-attention: nguồn với nguồn, mask PAD nguồn.

Decoder self-attention: đích với đích, causal mask.

Cross-attention: query từ đích, key và value từ nguồn, mask PAD nguồn.

Điểm cross-attention một head: `(Ly, Lx)`.

Mỗi hàng là một vị trí đích; softmax theo trục nguồn.

Cross-attention không cần causal mask: nguồn đã có đủ.

HỎI NHANH

Decoder có 7 vị trí đích, nguồn có 10 token. Ma trận trọng số cross-attention của một head, sau softmax, có shape gì, và cộng bằng 1 theo trục nào?

A `(10, 7)`, mỗi hàng là một token nguồn và cộng bằng 1

B `(7, 10)`, mỗi hàng là một vị trí đích và cộng bằng 1

C `(7, 10)`, mỗi cột là một token nguồn và cộng bằng 1

D `(7, 7)`, tam giác dưới vì decoder cần causal mask

HỎI NHANH

Một model dịch tự cài dịch riêng một câu nguồn 3 token thì ra một kết quả. Ghép đúng câu đó vào batch với một câu nguồn 30 token, câu ngắn được pad bên phải, thì bản dịch đổi. Nên nghi chỗ nào trước tiên?

A Mask PAD của nguồn chưa được đưa vào cross-attention

B Causal mask của decoder bị đảo phần trên và phần dưới

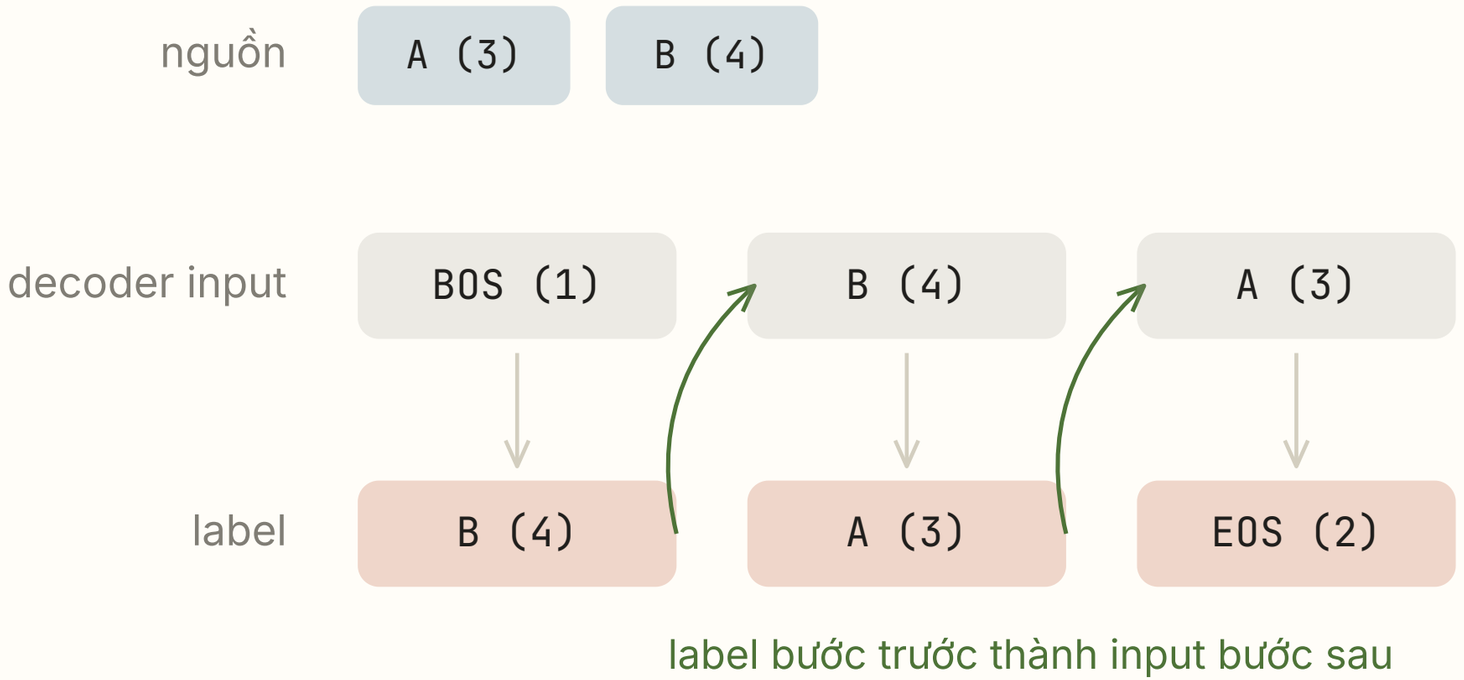
C Batch dài hơn nên softmax chạy với độ chính xác thấp hơn

D Encoder thiếu causal mask giống như decoder vẫn dùng

02

Teacher forcing và shift

Target B A EOS



Teacher forcing cho cặp A B của bài (ID từ code): decoder input BOS B A, label B A EOS; token đúng của bước trước được đưa làm input bước sau.

Train và inference khác tiền tố

Train: token đúng ở bước trước.

Inference: token model vừa sinh.

Một lỗi đầu làm tiền tố lệch, rồi lỗi có thể lan tiếp.

Đo cả teacher-forced loss và generation.

HỎI NHANH

Một đội muốn lúc train giống hệt lúc sinh. Ở mỗi bước, decoder GRU nhận token model vừa đoán bằng argmax thay cho token đích đúng, còn loss vẫn so với label đúng. Ở đầu quá trình train, chuyện gì xảy ra?

A Học nhanh hơn teacher forcing, vì lúc train đã giống hệt lúc sinh

B Học chậm và bất ổn, vì tiền tố gần như toàn token sai

C Không khác teacher forcing, vì loss vẫn so với label đúng

D Không train được, vì argmax chặn hết gradient tới tham số

03

Một seq2seq nhỏ chạy thật

Đảo chuỗi hai ký hiệu A, B, C

9 cặp có thể có.

7 cặp train; giữ C B và C C để thử.

GRU train từ đầu, chưa có attention.

Trạng thái cuối của encoder (như 3.8 đọc tại `lengths - 1`) làm trạng thái đầu của decoder.

Đảo chuỗi có oracle: mọi lỗi đều kiểm được.

Một cặp có thể kiểm bằng tay

A B → B A EOS

Nguồn: `[3, 4]`

Input đích: `[1, 4, 3]`

Label: `[4, 3, 2]`

BOS có ID 1, EOS có ID 2.

Tensor nguồn `(7, 2)`

Input và target `(7, 3)`

Logits `(7, 3, 6)`

HỎI NHANH

Logits có shape `(7, 3, 6)`. Muốn đưa vào cross-entropy thì flatten thành shape gì?

A `(126,)`

B `(7, 18)`

C `(6, 21)`

D `(21, 6)`

Loss có đáp án ở bước trước

KẾT QUẢ

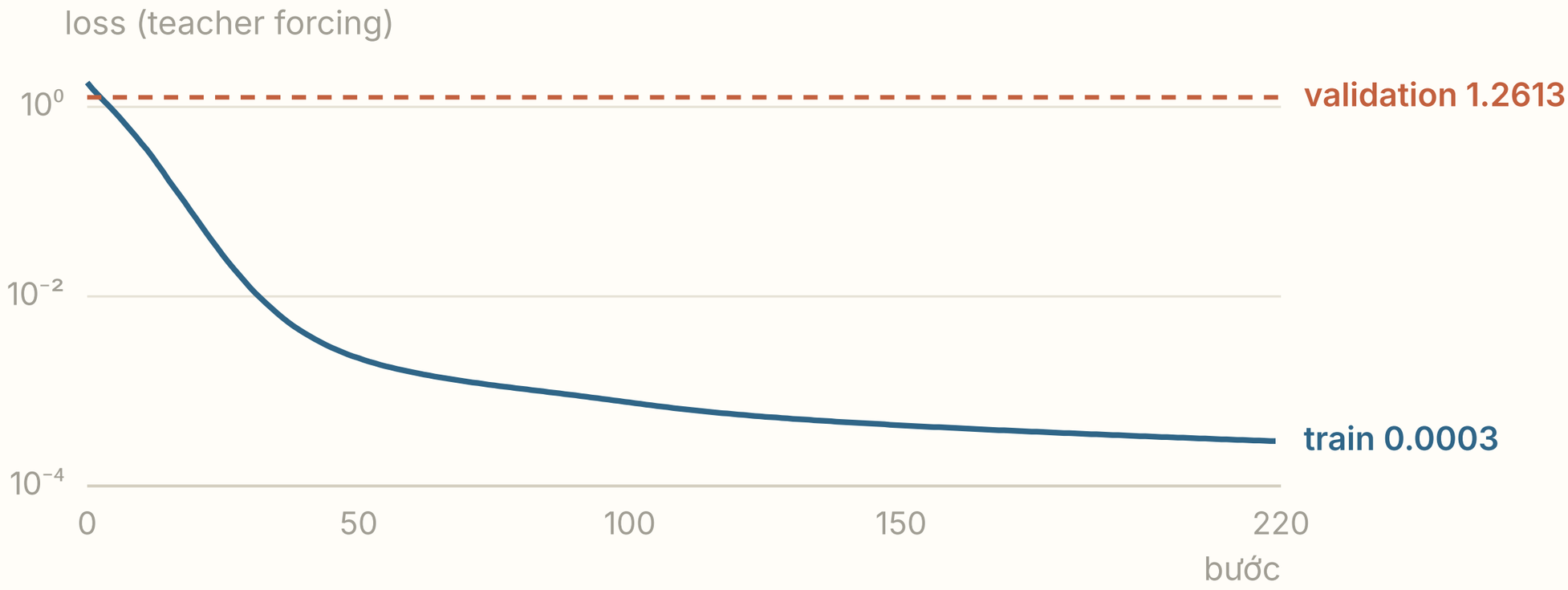
```
train loss đầu/cuối: 1.8026 0.0003
validation teacher-forced loss: 1.2613
```

Train loss 0.0003: khớp 7 cặp.

Validation 1.2613: hai tổ hợp chưa thấy còn yếu.

Vẫn có tiền tố đúng: chưa biết chuỗi tự sinh.

Loss qua 220 bước



Train loss của GRU seq2seq qua 220 bước (số từ code bài, trục log) xuống 0.0003, trong khi validation teacher-forced loss trên hai cặp chưa thấy vẫn là 1.2613.

Tự sinh làm lộ giới hạn

Train exact match: 7/7.

Validation exact match: 1/2.

C B → B C: đúng.

C C → C A: sai.

Model nhớ cặp train chưa chắc học quy tắc đảo.

Lỗi nằm ở token nào?

KẾT QUẢ

```
NLL từng vị trí: [[0.027, 0.729, 0.003], [0.002, 6.804, 0.003]]
C C, bước 2 sau BOS C: {'A': 0.994, 'B': 0.0, 'C': 0.001}
```

- Trung bình sáu NLL là 1.2613; riêng C C bước 2 là 6.804, gần 90%.
- C B bước 2: xác suất đúng khoảng 0.48, model phân vân.
- C C: tiền tố đúng BOS C mà vẫn đặt 0.994 vào A.

NLL từng vị trí

	vị trí 1	vị trí 2	EOS
C B	0.027	0.729	0.003
C C	0.002	6.804	0.003

C C, bước 2 sau BOS C:

A 0.994 C 0.001

tiền tố đúng vẫn chọn A

trung bình sáu ô bằng 1.2613; riêng 6.804 chiếm gần 90% tổng

NLL từng vị trí trên hai cặp validation (số từ code bài): lỗi dồn vào vị trí thứ hai của C C, nơi model đặt 0.994 vào A dù được cho tiền tố đúng BOS C.

HỎI NHANH

Ở mẫu C C, model sai token thứ hai cả khi free-running lẫn khi được cho tiền tố đúng BOS C. Đây là loại lỗi gì?

A Exposure bias, vì tiền tố tự sinh lệch khỏi tiền tố lúc train

B Lỗi giải mã, vì beam search rộng hơn sẽ tìm ra chuỗi C C

C Lỗi đo, vì exact match quá khắt khe với chuỗi hai token

D Lỗi tổng quát hóa, vì có tiền tố đúng mà model vẫn chọn A

04

Giải mã chuỗi đích

Beam search trên GRU vừa train

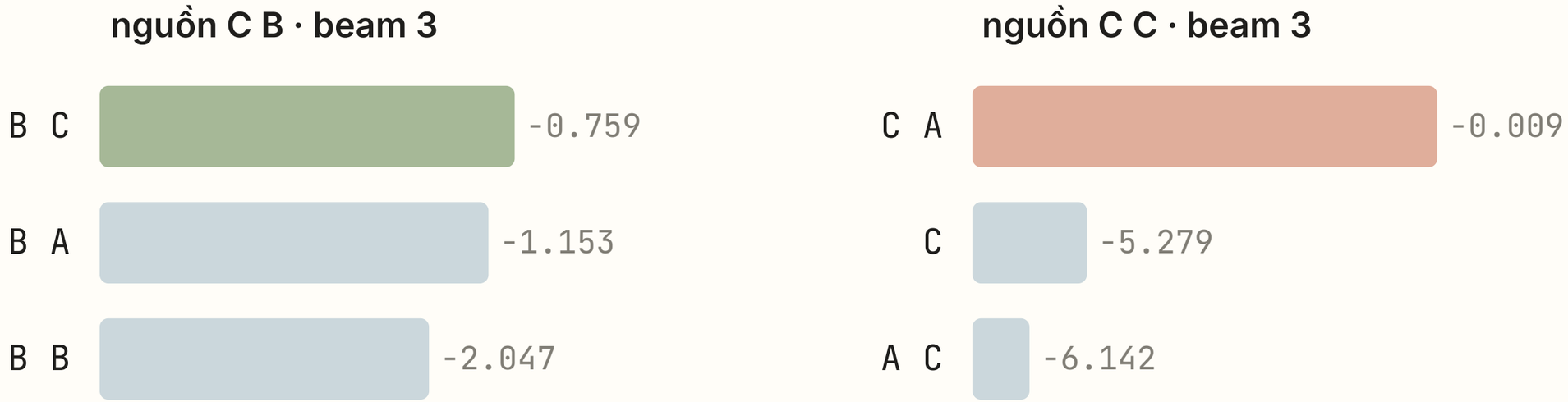
```
for pair in valid_pairs:
    print(" ".join(pair), "->", beam_search(pair)[:3])
```

KẾT QUẢ

```
C B -> [('B C', -0.759), ('B A', -1.153), ('B B', -2.047)]
C C -> [('C A', -0.009), ('C', -5.279), ('A C', -6.142)]
```

- Beam rộng 3: giữ 3 tiền tố có tổng log-prob cao nhất.
- C C cho C A với xác suất khoảng 0.991: beam tìm đúng lỗi của model.
- `C` dừng sớm vẫn lọt top 3 của beam.

Ba ứng viên của beam



tổng log-prob; C C không có trong top 3: beam chỉ tìm lỗi model tin chắc

Beam rộng 3 trên hai nguồn validation (số từ output bài): C B ra đúng B C với −0.759; C C ra C A với −0.009, còn đáp án C C không lọt vào top 3.

Tổng log-prob thiên về câu ngắn

$$\text{score}(y) = \frac{1}{|y|^{\alpha}} \sum_{t} \log p(y_t \mid y_{<t}, x)$$

$\alpha = 0$: tổng thô. $\alpha = 1$: trung bình mỗi token.

Hugging Face: `length_penalty`, mặc định 1.0.

HỎI NHANH

Có hai giả thuyết đã gặp EOS. `X EOS` có tổng log-prob -1.2, gồm 2 token. `X Y Z EOS` có -2.0, gồm 4 token. Xếp theo tổng thô và theo trung bình mỗi token, với $\alpha = 1$, thì chuỗi nào thắng?

A Theo tổng thô chuỗi dài thắng, theo trung bình chuỗi ngắn thắng

B Theo tổng thô chuỗi ngắn thắng, theo trung bình chuỗi dài thắng

C Cả hai cách xếp đều chọn chuỗi ngắn

D Cả hai cách xếp đều chọn chuỗi dài

Chặn lặp khi sinh

- `no_repeat_ngram_size=n`: logit của token tạo n-gram lặp thành $-\infty$.
- `repetition_penalty`: giảm mềm logit của token đã xuất hiện.
- Cả hai đổi phân phối model; chọn cùng beam size và `max_new_tokens` trên validation.

HỎI NHANH

Bản tóm tắt tin bóng đá bị lặp cụm “đội tuyển Việt Nam”, nên đội bật `no_repeat_ngram_size=2`, với mỗi âm tiết là một token. Nhưng bản tóm tắt đúng lại cần nhắc “Việt Nam” hai lần. Chuyện gì xảy ra?

A Không sao, vì luật này chỉ chặn hai token giống nhau đứng liền nhau

B Không sao, vì luật này chỉ trừ nhẹ điểm của token đã xuất hiện

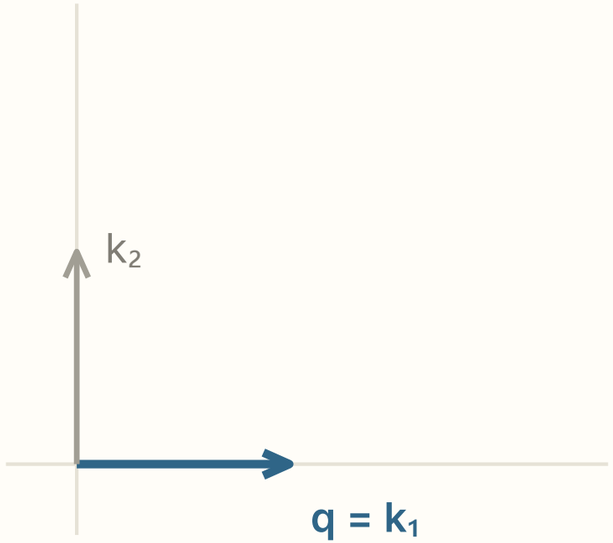
C Lần nhắc thứ hai bị chặn, nên model buộc phải viết theo cách khác

D Từ đó mọi token đã xuất hiện, kể cả “Việt”, đều bị chặn

05

Đọc lại nguồn bằng attention

Cross-attention gom thông tin từ nguồn



điểm (0.707, 0)

softmax (0.67, 0.33)

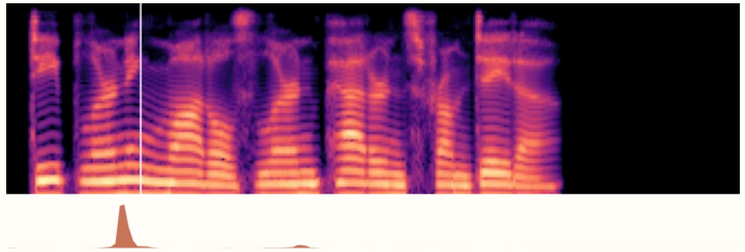
0.67·(2, 0) + 0.33·(0, 2)

context (1.34, 0.66)

vector đọc được là tổ hợp có trọng số của các value

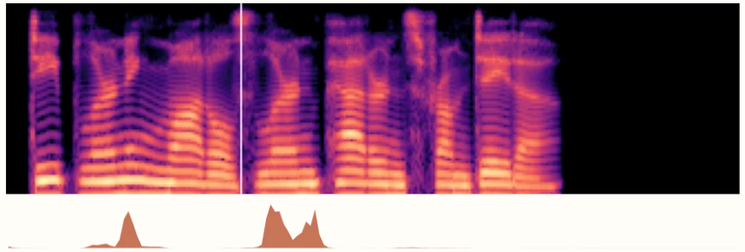
Phép tính tay của bài: query (1, 0) gần key thứ nhất hơn, softmax cho trọng số 0.67 và 0.33, context là tổ hợp (1.34, 0.66) của hai value (2, 0) và (0, 2).

Cross-attention của Whisper



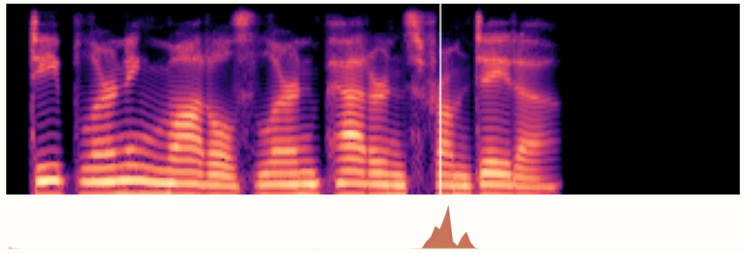
Deep learning
token đang sinh: “learning”

token 2/8: “learning”, trọng số lớn nhất ở 0.52 s



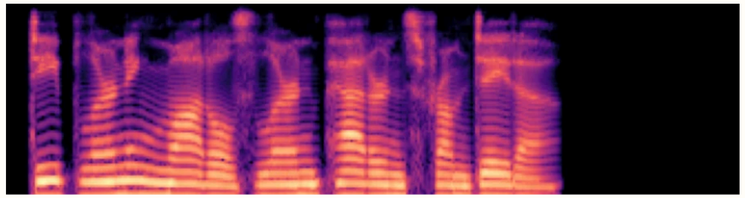
Deep learning models learn
token đang sinh: “learn”

token 4/8: “learn”, trọng số lớn nhất ở 1.18 s



Deep learning models learn patterns from
token đang sinh: “from”

token 6/8: “from”, trọng số lớn nhất ở 1.98 s



Deep learning models learn patterns from data.

transcript: “Deep learning models learn patterns from data.”

Whisper chép một câu đọc thật: encoder đọc phổ âm thanh; mỗi token decoder sinh ra, một head cross-attention (lớp 3, head 1) dồn trọng số vào đoạn âm của từ ấy.

Cross-attention tính tay

Query `(1,0)`; hai key `(1,0)`, `(0,1)`.

Hai value `(2,0)`, `(0,2)`.

Chia điểm cho $\sqrt{d_k} = \sqrt{2}$ trước softmax.

KẾT QUẢ

```
attention: [[0.67 0.33]]
context: [[1.34 0.66]]
```

Query gần key 1 nên value 1 nhận 0.67.

HỎI NHANH

Một query đích có điểm, đã chia cho $\sqrt{d_k}$, với ba key nguồn là $(0,\ \ln 3,\ \ln 3)$. Ba value là $(4, 0)$, $(0, 6)$ và $(9, 9)$, trong đó key thứ ba là PAD. Context khi có mask PAD và khi quên mask lần lượt là bao nhiêu?

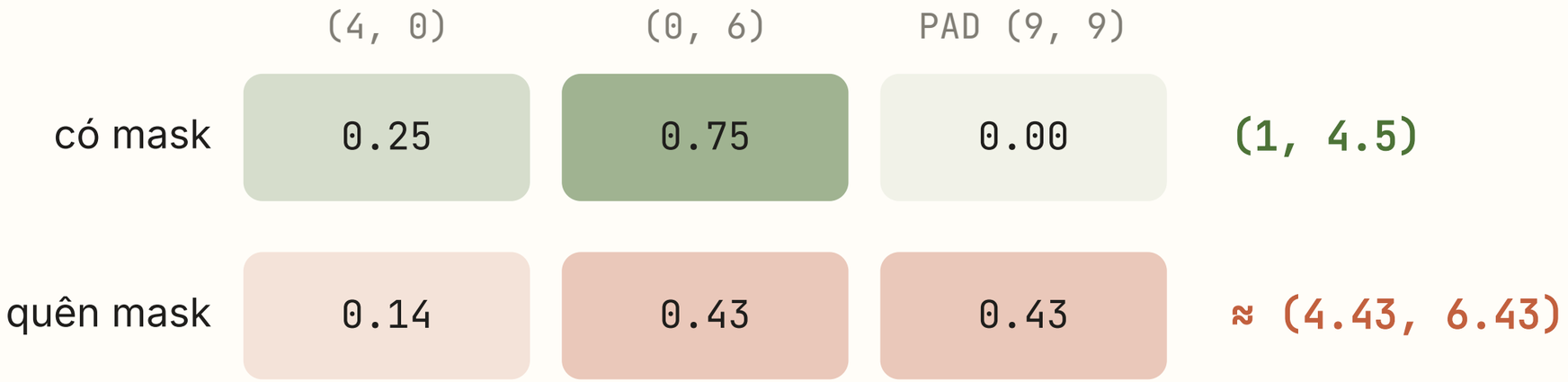
A $(1,\ 4.5)$ và khoảng $(4.43,\ 6.43)$

B $(1,\ 4.5)$ ở cả hai trường hợp

C Khoảng $(4.43,\ 6.43)$ và $(1,\ 4.5)$

D $(4,\ 6)$ và $(13,\ 15)$

PAD kéo context đi xa



value của PAD kéo context đi xa khi mask bị quên

Câu hỏi về mask PAD: điểm (0, ln 3, ln 3) cho trọng số (0.25, 0.75) khi mask key PAD, context (1, 4.5); quên mask thì trọng số (1, 3, 3)/7 và context bị kéo tới khoảng (4.43, 6.43).

06

Chấm chuỗi sinh ra

BLEU tới bigram trên một câu

Tham chiếu: `câu lạc bộ họp vào thứ sáu`

KẾT QUẢ

```
họp họp họp họp              p1=0.250 p2=0.000 BP=0.472 BLEU-2=0.000
câu lạc bộ họp thứ sáu       p1=1.000 p2=0.800 BP=0.846 BLEU-2=0.757
câu lạc bộ họp vào thứ bảy   p1=0.857 p2=0.833 BP=1.000 BLEU-2=0.845
```

$\mathrm{BP} = \min(1, e^{1-r/c})$. BLEU chuẩn dùng tới 4-gram, gộp trên cả corpus.

Clipping, BP và sự thật

- Lặp `họp`: chỉ khớp 1 trên 4, không bigram nào khớp, điểm 0.
- Đúng nội dung nhưng ngắn: BP 0.846, điểm 0.757.
- Sai ngày: 0.845, cao hơn câu đúng.
- BLEU phụ thuộc tokenizer; dùng đúng phiên bản đề cho.

Ba câu, ba điểm BLEU-2

tham chiếu: câu lạc bộ họp vào thứ sáu

	p1	p2	BP	BLEU-2
họp họp họp họp	0.250	0.000	0.472	**0.000**
câu lạc bộ họp thứ sáu	1.000	0.800	0.846	**0.757**
câu lạc bộ họp vào thứ bảy	0.857	0.833	1.000	**0.845**

câu sai thứ bảy được điểm cao hơn câu đúng: overlap n-gram không kiểm sự thật

BLEU-2 của ba câu (số từ output bài): clipping đưa câu lặp “họp” về 0; câu đúng nhưng thiếu “vào” bị brevity penalty còn 0.757; câu sai thứ bảy vẫn được 0.845.

Metric phải theo nhiệm vụ

- Dịch: BLEU, precision n-gram có clipping và BP.
- Tóm tắt: ROUGE-N là recall n-gram; ROUGE-L theo subsequence chung.
- Recall không phạt bản dài: cần giới hạn độ dài hoặc F1.
- Không metric nào tự kiểm ngày, số, phủ định hay vật có trong ảnh.

HỎI NHANH

Đề tóm tắt chấm ROUGE-1 recall, không giới hạn độ dài. Một đội nộp nguyên văn bản nguồn làm “tóm tắt”, và đứng đầu bảng xếp hạng tạm. Điều này cho thấy gì?

A Bản nguồn là tóm tắt tốt nhất, vì nó chứa đủ mọi thông tin

B ROUGE đã hỏng, vì nó không chạy đúng với văn bản dài

C Recall không phạt độ dài, nên metric cần giới hạn độ dài hoặc dùng F1

D Đội đó đã đọc trộm bản tham chiếu của tập test

FLAN-T5 small qua recipe

- Snapshot local; kiểm repo ID, revision, SHA-256 từng tệp.
- Prompt tối đa 256 token; dài hơn thì dừng, không tự chia đoạn.
- `generate` greedy, `--max-new-tokens` mặc định 48.
- In JSON: output, thời gian, số token, bộ nhớ đỉnh (trên CUDA).
- Thử beam, `length_penalty`: thêm vào `generate`, so trên validation.

07

Thư viện và khi đi thi

Thư viện tự dịch label

```
labels = torch.tensor([[4, 3, 1, -100], [5, 1, -100, -100]])
t5.prepare_decoder_input_ids_from_labels(labels=labels)
```

KẾT QUẢ

```
decoder input: [[0, 4, 3, 1], [0, 5, 1, 0]]
vị trí tính loss: 5
label đã có token mở đầu: [[0, 0, 4, 3], [0, 0, 5, 1]]
```

T5: PAD (ID 0) cũng là token mở đầu, EOS có ID 1. Label có sẵn token mở đầu sẽ bị dịch hai lần.

Khi đi thi

- Metric quyết định giải mã: BLEU phạt ngắn, recall không phạt dài.
- In nguồn, input và label: EOS cuối label, PAD là `-100`.
- Chọn checkpoint bằng output tự sinh, không bằng loss.
- Dự toán thời gian sinh; đặt `max_new_tokens`.
- Đọc vài chục output: lặp, phủ định, số, ngày.

08

Luyện đề

LUYỆN ĐỀ 1

Một model tóm tắt encoder–decoder đọc nguồn 4000 token và sinh tóm tắt 100 token, mọi attention đều đầy đủ. Nếu nguồn tăng lên 8000 token mà tóm tắt vẫn 100 token, thì ma trận điểm của encoder self-attention và của cross-attention lớn lên bao nhiêu lần?

A Cả hai lớn lên 2 lần

B Cả hai lớn lên 4 lần

C Encoder self-attention 4 lần, cross-attention 2 lần

D Encoder self-attention 2 lần, cross-attention 4 lần

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 230

LUYỆN ĐỀ 2

Một model sinh caption gồm image encoder biến ảnh thành 196 vector patch, và một text decoder sinh caption. Một bạn đề xuất bỏ cross-attention cho nhẹ, decoder chỉ giữ causal self-attention trên các token chữ. Hậu quả chính là gì?

A Không đổi nhiều, vì self-attention trên chữ vẫn thấy được các patch ảnh

B Vẫn bám ảnh, vì thông tin ảnh đã nằm sẵn trong embedding chữ

C Decoder mất đường đọc ảnh, chỉ còn là một language model trên chữ

D Model không train được, vì causal mask phải đi kèm cross-attention

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 265

LUYỆN ĐỀ 3

Một model dịch dựng như GRU trong bài: encoder dồn cả câu nguồn vào một vector 24 số rồi giao cho decoder. Với câu nguồn trên 40 token, bản dịch hay bỏ sót ý ở đầu câu. Thêm cross-attention vào decoder giúp đúng ở chỗ nào?

A Mỗi bước sinh đọc lại mọi vị trí nguồn, theo trọng số riêng

B Nén cả câu nguồn thành một vector cố định giàu thông tin hơn

C Cho decoder nhìn trước các token đích chưa sinh của bản dịch

D Cho decoder sinh mọi token đích song song, không chờ tiền tố

PHỎNG THEO NOAI 2025 Singapore · Phần 1 · Câu 16

Chốt ý

Nguồn đã có; đích sinh từ tiền tố; cross-attention đọc nguồn.

Teacher forcing khác free-running; tách loss theo vị trí.

Beam tìm chuỗi tốt theo model; BLEU và ROUGE chưa kiểm sự thật.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; thử beam rộng 1 và 3 trên C B, C C, rồi đối chiếu với oracle đảo chuỗi.